

导师汇报材料：Stage3/Stage4 风场状态构建与实验收敛

时间：2026-05-12

一、项目作用和最终目的

这个项目要解决的是航空场景中的局部风场感知问题。实际飞行环境里，风场对飞行安全、航路规划、颠簸风险和航空气象态势判断都很重要，但风不是每个位置都能直接测到的。现有数据往往来自飞机轨迹、少量直接风观测、运动状态和不同时间片的信息，具有明显的稀疏、多源、不连续特点。

因此，本项目的作用可以概括为一句话：

把零散的航空观测线索融合成稳定、可解释、可用于后续预测的三维风场状态层。

更通俗地说，项目不是简单画一张风场图，而是做一个“风场状态底座”：先把原始数据整理成统一网格，再识别每一帧有哪些飞机、飞机之间是否能协同感知，最后把直接风观测、轨迹运动线索、时序背景和通信图信息融合起来，形成后续模型可以直接使用的风场状态。

项目最终目的分为两个层次：

- 当前阶段目标：完成 Stage4 风场状态构建层，使其稳定、可运行、可解释、可复现，并能支撑论文中的主结果和消融实验。
- 后续阶段目标：基于 Stage4 输出进入 Stage5，实现事件驱动、ROI 聚焦、短时实时风场预测，为飞行风险预警和辅助决策提供输入。

所以当前工作的重点不是盲目追求更高 coverage，也不是马上训练一个端到端预测网络，而是先把“稀疏航空观测 -> 可靠风场状态层”这一步做扎实。这个状态层后续既可以作为论文方法的核心，也可以作为实时预测和风险判断的输入基础。

二、当前研究定位

当前工作聚焦于“面向空地一体协同感知的稀疏多源风场状态构建”。现阶段重点不是直接训练一个端到端预测网络，而是先把稀疏、多源、时序不连续的航空风观测整理成稳定、可解释、可运行的三维风场状态层。

当前系统分为三个主要阶段：

- Stage1：完成原始观测数据整理、字段清洗和基础对齐。
- Stage2：完成体素化，输出 stage2_summary.json 与逐帧 voxel npz。
- Stage3：构建 flight agents 和 communication graph，输出逐帧 agents JSON。
- Stage4：融合多源信息进行风场重建，并导出后续训练或预测可用的状态层。

后续 Stage5 将在 Stage4 状态层基础上单独设计，方向是事件驱动、ROI 聚焦、短时实时风场预测。

三、前期阶段和主要工作

前期工作已经完成了从原始航空观测到风场状态层的基础链路，不只是当前这一周的 Stage4 调参。

1. Stage1：原始数据整理

Stage1 主要负责把原始观测数据整理成后续可处理的统一格式，重点包括：

- 清洗原始飞行观测字段。
- 对时间、空间、高度等基础字段做统一。
- 保留后续风场重建需要的飞行轨迹、速度、位置和观测相关信息。

这一阶段的作用是减少原始数据格式差异对后续流程的影响，让后续 Stage2 可以稳定进行体素化。

2. Stage2：体素化与逐帧样本构建

Stage2 将原始观测映射到统一的三维时空网格中，形成逐帧 voxel 表示。主要输出包括：

- stage2_summary.json
- 逐帧 voxel npz

这一阶段解决的是“稀疏、多源、时序不连续观测如何进入统一网格”的问题，是 Stage3 和 Stage4 的共同输入。

3. Stage3: Agent 与通信图构建

Stage3 基于 Stage2 的逐帧 voxel 和飞行信息构建 flight agents，并进一步计算通信候选关系。主要工作包括：

- 生成每一帧的 flight agents。
- 统计有效 flight agent 和 wind-capable flight。
- 构建 flight-flight motion edges。
- 构建 flight-wind 相关 edges。
- 形成 communication allowed agents / edges。
- 输出 agents/*.json 和 stage3_summary.json。

这一阶段对应协同感知中的 agent / communication graph 层，为后续 Where2Comm 风格的选择和多机协同重建提供结构信息。

4. Stage4 早期主链路：多源风场重建

Stage4 早期版本已经打通了多源风场重建主流程，融合的信息包括：

- direct wind observation
- flight trajectory
- motion-derived cues
- seed / support information
- temporal background
- communication graph information

这一阶段的核心输出是三维风场状态，包括：

- recon_u_3d
- recon_v_3d
- recon_confidence_3d
- recon_mask_3d
- physics_weight_3d
- communication / support / auxiliary fields

早期版本解决了“能不能从稀疏多源观测构建风场状态层”的问题。

5. Stage4 收敛阶段：速度、质量和可复现性

最近阶段的工作重点从“打通流程”转向“收敛为论文实验平台”。主要包括：

- 引入 fast 主链，减少不必要 dense auxiliary 输出。
- 引入 GPU 可选数值核，但不强行把整条链路搬到 GPU。
- 优化 confidence 和 support 策略，提升状态层质量。
- 将 full_aux_export 改成真正基于 fast 结果补写辅助场。
- 明确 Stage3 可分片、Stage4 默认不分片。
- 增加模块级开关，为后续 S0-S6 消融做准备。

6. Stage5 后续方向

Stage5 目前尚未正式进入实现阶段。后续不计划做“每帧全域预测”，而是基于 Stage4 状态层做：

- 事件驱动触发。
- ROI 聚焦。
- 短时风场预测。
- 面向飞行风险的辅助决策。

这样可以避免把 Stage5 过早塞进 Stage4，保持论文结构和工程职责清晰。

四、当前真实代码和目录关系

当前真实运行使用 v2 主线：

- 主控脚本: `run_stage34_workflow_v2.sh`
- Stage3 代码: `stage3_agents_v2.py`
- Stage4 代码: `stage4_pack_v2.py`

旧版 `stage3_agents.py` 和 `stage4_pack.py` 不作为当前主线。

当前输入输出关系如下:

```
stage2_output/  
  stage2_summary.json  
  frame/voxel npz  
    -> stage3_agents_v2.py  
    -> stage3_output_v2/  
        stage3_summary.json  
        agents/*.json  
    -> stage4_pack_v2.py  
    -> stage4_output_v2/  
        stage4_summary.json  
        frame/*.npz  
    -> full_aux_export  
    -> stage4_output_full_aux_v2/<RUN_LABEL>/
```

关键目录说明:

目录/变量	含义
STAGE2_OUTPUT_DIR	Stage2 输出, 默认 <code>\$BASE_DIR/stage2_output</code>
STAGE3_OUTPUT_DIR_V2	当前 Stage3 正式输出, 默认 <code>\$BASE_DIR/stage3_output_v2</code>
STAGE3_INPUT_DIR_ORIG	旧 Stage3 输入目录, 默认 <code>\$BASE_DIR/stage3_output</code>
STAGE4_OUTPUT_DIR	当前 Stage4 fast 主输出, 默认 <code>\$BASE_DIR/stage4_output_v2</code>
STAGE4_RUN_ROOT	当前 run 的 Stage4 子集目录, 默认 <code>\$BASE_DIR/stage4_output_runs_v2</code>
STAGE4_FAST_SOURCE_DIR	FullAux 导出的 fast 源目录, 默认读取 <code>stage4_output_v2</code>
STAGE4_FULL_AUX_OUTPUT_ROOT	FullAux 输出根目录, 默认 <code>\$BASE_DIR/stage4_output_full_aux_v2</code>

注意: 单独运行 `stage4_only` 时, 需要显式指定:

```
STAGE3_INPUT_DIR_FOR_STAGE4=$BASE_DIR/stage3_output_v2
```

否则可能误读旧目录 `stage3_output`。

五、核心工程结论

1. Stage3 可以并行分片

Stage3 主要做每帧 agent pack 和通信图构建, 任务接近 frame-level 独立, 主要消耗 CPU / IO / Polars, 因此适合分片并行。

当前推荐设置:

```
STAGE3_PARALLEL_SHARDS=8  
STAGE3_CPU_THREADS_PER_WORKER=1
```

2. Stage4 默认不能多进程分片

Stage4 不是逐帧完全独立的重构程序, 内部依赖前一帧状态, 例如:

- `temporal_background`
- `forecast_next_wind_field`
- `trigger_reconstruction`
- `prev_recon_state`

如果多进程按帧分片，会切断时序链，导致 shard 首帧变成 `first_frame` 或 `discontiguous_frame_gap`，从而改变重构结果。因此当前主线策略是：

```
MULTI_GPU_STAGE4_SHARD=0
STAGE4_CPU_THREADS=6
WIND_STAGE4_USE_GPU=1
WIND_STAGE4_GPU_DEVICE=cuda:0
```

也就是 `Stage4` 单进程单卡串行，CPU 内部用多线程加速。

3. Stage4 进入冻结准备

冻结不是表示不再修 bug，而是表示不再继续无边界追求更高 coverage，也不再引入复杂 heuristic。当前冻结原则是：

- anchor fidelity 稳定优先。
- coverage 只追求温和提升。
- confidence 要保持层次性，不追求单纯抬高平均值。
- 主链路要可复现、可消融、可全量运行。
- 后续新增预测能力放到 Stage5，不继续塞进 Stage4。

六、Stage4 当前工作内容

1. Fast / FullAux 双分支设计

当前 `Stage4` 分成两个输出 profile。

`S5 FinalFast`：

- 作为正式运行主版本。
- 输出 profile: `fast`。
- 用于全量重构、论文主结果、Stage5 的主状态输入。
- 保留核心重构字段，不默认输出所有 dense auxiliary fields。

`S6 FullAux`：

- 作为训练前导出版本。
- 输出 profile: `full_aux_export`。
- 从 `S5 FinalFast` 结果读取 `recon_*`，只补 richer auxiliary fields。
- 不改变主重构结果。

2. Fast 主链保留字段

`S5 FinalFast` 主要保留：

- `recon_u_3d`
- `recon_v_3d`
- `recon_confidence_3d`
- `recon_mask_3d`
- `trajectory_3d`
- `physics_weight_3d`
- communication 相关字段
- 稀疏观测与 flight graph

3. FullAux 补充字段

S6 FullAux 主要补写：

- pinn_divergence_3d
- pinn_smoothness_3d
- hazard_*
- diffusion_condition_4d
- direct_agreement_3d
- direct_source_count_3d
- source_diversity_3d

它的定位是训练前样本增强，不是运行主链版本。

七、主要参数说明

这一节专门解释命令里每个常见参数是什么意思。可以把这些参数理解为“开关”和“路径”：有些决定跑哪一段，有些决定读写哪个目录，有些决定开不开某个算法模块。

1. 参数命名怎么看

- RUN_*：控制这次实验怎么跑，例如跑全量、跑哪一阶段、日志叫什么名字。
- STAGE*_DIR：控制输入输出目录。
- STAGE*_CPU_THREADS / STAGE3_PARALLEL_SHARDS：控制并行和 CPU 使用。
- WIND_STAGE4_*：控制 Stage4 具体算法、GPU、输出 profile 和模块开关。
- OMP_NUM_THREADS / MKL_NUM_THREADS / NUMEXPR_NUM_THREADS / POLARS_MAX_THREADS：底层数学库和 Polars 的线程数。

参数值里常见的 1/0 含义是：

- 1：开启。
- 0：关闭。

2. 主运行参数

参数	当前推荐值	说明
RUN_MODE	full	选择数据范围。full 表示跑全量；first3 表示只跑前 3 帧；offset 表示从某个 offset 开始跑一段；indices 表示指定若干帧；topwind_auto 表示自动挑风相关较强的帧。
RUN_PHASE	full_fast_multi_gpu	选择跑哪条流程。当前主线用 full_fast_multi_gpu，含义是 Stage3 分片并行，Stage4 仍然单卡串行。
RUN_PHASES	空	更细粒度指定阶段，例如只跑 stage3,stage4,validate。一般不用，除非调试。
RUN_LABEL_OVERRIDE	full_fast_stage4_frozen_v1	自定义这次运行的名字。日志和输出记录都会带这个标签，方便以后找结果。
RUN_VALIDATE	1	是否跑基础校验。1 表示跑，能及时发现缺文件、目录错、summary 不存在等问题。
PROGRESS_EVERY	50	每隔多少帧打印一次进度。全量运行建议设大一点，日志更清楚。
FAST_FULL_MODE	通常不用手动设	RUN_PHASE=full 时可启用 fast 模式；当前推荐直接使用 RUN_PHASE=full_fast_multi_gpu。
FRAME_OFFSET	按需	RUN_MODE=offset 时使用，表示从第几帧开始。
MAX_FRAMES	按需	子集实验时使用，限制最多跑多少帧。

FRAME_INDICES	按需	RUN_MODE=indices 时使用，手动指定帧编号列表。
---------------	----	---------------------------------

3. 目录参数

参数	当前默认值	说明
BASE_DIR	/data/LFT-W02_data/pengxu	数据和结果的根目录。
STAGE_DIR	\$BASE_DIR/stage	代码目录，主控脚本在这里。
STAGE2_OUTPUT_DIR	\$BASE_DIR/stage2_output	Stage2 的输出目录，里面有 stage2_summary.json 和逐帧 voxel npz。
STAGE3_OUTPUT_DIR_V2	\$BASE_DIR/stage3_output_v2	当前 Stage3 正式输出目录。
STAGE3_INPUT_DIR_ORIG	\$BASE_DIR/stage3_output	旧版 Stage3 输出目录，只作为兼容默认值，不建议当前主线使用。
STAGE3_INPUT_DIR_FOR_STAGE4	建议手动设为 \$BASE_DIR/stage3_output_v2	单独跑 stage4_only 时一定要设，避免误读旧目录。
ACTIVE_STAGE3_DIR	脚本自动决定	Stage4 实际读取的 Stage3 目录。跑完整链路时通常是 stage3_output_v2。
STAGE4_OUTPUT_DIR	\$BASE_DIR/stage4_output_v2	S5 FinalFast 的正式输出目录。
STAGE4_RUN_ROOT	\$BASE_DIR/stage4_output_runs_v2	每次运行的结果副本/子集目录根路径。
STAGE4_RUN_DIR	\$STAGE4_RUN_ROOT/\$RUN_LABEL	当前这次 run 对应的 Stage4 子目录。
STAGE4_FAST_SOURCE_DIR	\$BASE_DIR/stage4_output_v2	S6 FullAux 补导出时读取的 fast 源结果目录。
STAGE4_FULL_AUX_OUTPUT_ROOT	\$BASE_DIR/stage4_output_full_aux_v2	S6 FullAux 输出根目录。
LOG_ROOT_DIR	\$STAGE_DIR/logs_v2	日志根目录。
LOG_DIR	脚本自动生成	当前 run 的具体日志目录。

4. Stage3 并行参数

参数	当前推荐值	说明
STAGE3_PARALLEL_SHARDS	8	Stage3 分成多少个 shard 并行跑。可以理解为把帧分成 8 份同时处理。
STAGE3_CPU_THREADS_PER_WORKER	1	每个 Stage3 worker 用多少 CPU 线程。8 个 worker 每个 1 线程，通常比少数 worker 抢很多线程更稳定。
WIND_STAGE3_OUTPUT_DIR	脚本自动设置	stage3_agents_v2.py 实际写出的目录。分片时会写到 shard 目录，合并后进入 stage3_output_v2。
STAGE3_SHARD_ROOT	脚本内部使用	Stage3 分片临时目录。
STAGE3_MERGED_DIR	脚本内部使用	Stage3 shard 合并后的目标目录。

5. Stage4 运行参数

参数	当前推荐值	说明
----	-------	----

MULTI_GPU_STAGE4_SHARD	0	是否把 Stage4 也分片。当前必须默认 0，因为 Stage4 有时序依赖。
STAGE4_CPU_THREADS	6	Stage4 单进程内部使用的 CPU 线程数。主要影响 Polars、NumPy、BLAS 等 CPU 计算。
WIND_STAGE4_USE_GPU	1	是否启用 GPU。当前只是部分 dense 数值核用 GPU，不是整条链路都在 GPU 上。
WIND_STAGE4_GPU_DEVICE	cuda:0	指定使用哪张 GPU。当前主线用单卡。
WIND_STAGE4_GPU_MIN_NUMEL	默认 1200000	张量元素数量超过这个阈值才走 GPU，避免小数组频繁搬运导致更慢。
WIND_STAGE4_FAST_MODE	脚本自动设为 1	开启 fast 主链模式，减少不必要的重辅助场输出。
WIND_STAGE4_OUTPUT_PROFILE	fast / full_aux_export	输出档位。fast 是主链结果，full_aux_export 是训练前补完整辅助场。
WIND_STAGE4_ENABLE_QUALITY_EXPAND	S5 为 0，S6 为 1	是否启用更强的质量扩展。主链保守，训练导出更完整。
WIND_STAGE4_FAST_SKIP_POST	fast 下通常为 1	fast 模式是否跳过一部分后处理。
WIND_STAGE4_FAST_SKIP_DENSE_AUX	fast 下通常为 1	fast 模式是否跳过完整 dense auxiliary fields。
WIND_STAGE4_SAVE_COMPRESSED	S5 通常 0，S6 通常 1	是否压缩 npz。fast 主链更重视速度，FullAux 更重视完整保存。
WIND_STAGE4_AUX_SOURCE_DIR	STAGE4_FAST_SOURCE_DIR	full_aux_export 实际读取 fast 源结果的目录。
WIND_STAGE4_OUTPUT_DIR	STAGE4_OUTPUT_DIR	stage4_pack_v2.py 实际写出的输出目录。
WIND_STAGE3_INPUT_DIR	ACTIVE_STAGE3_DIR	stage4_pack_v2.py 实际读取的 Stage3 输入目录。

STAGE4_CPU_THREADS=6 会统一影响：

- OMP_NUM_THREADS
- MKL_NUM_THREADS
- NUMEXPR_NUM_THREADS
- POLARS_MAX_THREADS

这四个底层线程变量的通俗含义是：

参数	含义
OMP_NUM_THREADS	OpenMP 并行线程数，影响部分底层数值计算。
MKL_NUM_THREADS	Intel MKL 数学库线程数，影响矩阵和数组计算。
NUMEXPR_NUM_THREADS	NumExpr 表达式计算线程数。
POLARS_MAX_THREADS	Polars DataFrame 计算线程数。

6. Stage4 模块开关

参数	含义
WIND_STAGE4_ENABLE_SUPPORT_FILL	是否启用 support fill。意思是根据轨迹、运动和邻域支持，把没有直接风观测但有可信支撑的位置补起来。
WIND_STAGE4_ENABLE_TEMPORAL_FILL	是否启用 temporal fill。意思是参考前一帧状态，补当前帧缺失区域，增强时序连续性。

WIND_STAGE4_ENABLE_RELAX	是否启用轻量物理一致性松弛。意思是让局部风场更平滑、更符合邻域一致性。
WIND_STAGE4_ENABLE_PRUNE	是否启用低质量区域裁剪。意思是把置信度或支撑不足的尾部区域清掉。
WIND_STAGE4_ENABLE_EXPAND	是否启用 support expand。意思是在高支撑区域做温和扩展，提高有效覆盖。
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_RESTORE	是否恢复 direct anchor。意思是如果直接观测点被后处理影响，要把它们恢复回来。
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_FORCE	是否强制保护 direct anchor。意思是最终强制直接观测锚点优先，防止重构结果偏离真实观测。
WIND_STAGE4_BASE_RECON_ENABLE_IDW	是否启用基础 IDW 插值。当前不是主线重点。
WIND_STAGE4_BASE_RECON_IDW_MAX_FILL	IDW 最多补多少点。用于限制基础插值规模。

S5 FinalFast 全部开启：

```
WIND_STAGE4_ENABLE_SUPPORT_FILL=1
WIND_STAGE4_ENABLE_TEMPORAL_FILL=1
WIND_STAGE4_ENABLE_RELAX=1
WIND_STAGE4_ENABLE_PRUNE=1
WIND_STAGE4_ENABLE_EXPAND=1
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_RESTORE=1
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_FORCE=1
```

7. FullAux 导出参数

S6 FullAux 需要额外指定：

```
RUN_PHASE=full_aux_export
STAGE4_FAST_SOURCE_DIR=$BASE_DIR/stage4_output_v2
```

脚本会将输出目录切换为：

```
stage4_output_full_aux_v2/<RUN_LABEL>
```

8. 报告和检查参数

参数	推荐值	说明
STAGE4_OUTLIER_TOPK	10	outlier 报告最多输出多少个异常点。默认调小，避免报告过重。
STAGE4_NPZ_CHECK_SAMPLES	1	npz 字段检查抽样多少帧。全量时不需要每帧都查。
STAGE4_READINESS_PY	按需	指定 readiness 报告脚本路径。一般不用手动设。

9. 参数重点理解版

如果不看代码，只需要先记住下面几组：

要理解的问题	对应参数	一句话解释
这次跑多少数据	RUN_MODE	full 是全量，offset/indices 是子集调试。
这次跑哪条流程	RUN_PHASE	当前主线是 full_fast_multi_gpu。
这次结果叫什么	RUN_LABEL_OVERRIDE	给实验起名字，之后查日志和结果靠它。

Stage3 怎么加速	STAGE3_PARALLEL_SHARDS=8	把 Stage3 分成 8 份并行跑。
Stage4 为什么不分片	MULTI_GPU_STAGE4_SHARD=0	Stage4 依赖上一帧，默认不能切断时序链。
Stage4 用多少 CPU	STAGE4_CPU_THREADS=6	Stage4 是单进程，但内部用 6 个 CPU 线程。
Stage4 用哪张 GPU	WIND_STAGE4_GPU_DEVICE=cuda:0	只让部分 dense 数值计算上 GPU。
当前主结果输出	WIND_STAGE4_OUTPUT_PROFILE=fast	对应 S5 FinalFast。
训练前补字段	WIND_STAGE4_OUTPUT_PROFILE=full_aux_export	对应 S6 FullAux。
控制模块开关	WIND_STAGE4_ENABLE_*	用来做 S0-S6 消融。

最重要的一句话是：

Stage3 可以切片并行，因为它基本按帧独立；Stage4 不能默认切片，因为它要用上一帧风场状态。

八、关键函数说明

这一节解释代码中主要函数的作用。可以把它们理解成流水线里的工人：主控脚本负责“调度”，Stage3 负责“生成 agent 和通信关系”，Stage4 负责“重建风场状态”。

1. run_stage34_workflow_v2.sh 中的主要函数

函数	通俗解释
log() / warn() / die()	打印普通日志、警告和错误。die() 会直接终止运行。
require_file()	检查必须存在的文件，比如 summary，不存在就报错停止。
sanitize_tag()	把运行标签里的特殊字符处理成适合做目录名/文件名的形式。
reset_phase_plan()	清空当前阶段计划。
enable_phase()	把某个阶段加入本次运行计划。
phase_enabled()	判断某个阶段这次是否需要跑。
enable_report_bundle()	开启报告相关阶段，例如 sparse metrics / outliers 等。
enable_full_bundle()	开启完整运行阶段组合。
enable_runtime_bundle()	开启当前默认轻量运行组合，通常包括 stage3、stage4、validate、collect、readiness。
enable_aux_runtime_bundle()	开启 FullAux 导出所需的运行组合。
enabled_phases_csv()	把已启用阶段整理成字符串，便于写日志。
configure_v2_stage_dirs()	根据当前 phase 决定 Stage3/Stage4 读写哪个目录。
plan_run_phases()	根据 RUN_PHASE 或 RUN_PHASES 规划这次要跑哪些阶段。
init_phase_status()	初始化阶段状态日志。
mark_phase()	记录某个阶段是 RUN、DONE、SKIP 还是 FAIL。
select_run_mode()	根据 RUN_MODE 选择跑全量、子集还是指定帧。
write_run_info()	把本次运行的参数、目录和标签写入日志，方便复盘。
run_stage3_script()	调用 stage3_agents_v2.py。
run_stage4_script()	调用 stage4_pack_v2.py。
run_full_fast_multi_gpu()	当前主链函数：Stage3 分片并行，合并后 Stage4 单卡串行。

<code>build_frame_indices_for_current_run()</code>	根据 <code>RUN_MODE</code> 生成要处理的帧编号列表。
<code>split_indices_into_shards()</code>	把帧编号切成多个 shard 给 Stage3 并行处理。
<code>merge_stage3_shards()</code>	把多个 Stage3 shard 的结果合并到 <code>stage3_output_v2</code> 。
<code>merge_stage4_shards()</code>	合并 Stage4 shard。当前不推荐默认使用，只保留实验能力。
<code>run_validate()</code>	跑基础校验，确认输入输出和 summary 是否一致。
<code>write_stage3_summary_logs()</code>	整理 Stage3 summary 到日志。
<code>write_stage4_summary_logs()</code>	整理 Stage4 summary 到日志。
<code>collect_current_stage4_run()</code>	把当前 Stage4 结果整理到 run 子目录，方便保存和区分实验。
<code>run_export()</code>	导出训练或后处理格式。当前默认主链不强制跑。
<code>run_stage4_readiness_report()</code>	检查 Stage4 输出是否适合后续训练/使用。
<code>run_stage4_sparse_metrics_report()</code>	计算 sparse anchor fidelity 指标。论文需要时再跑。
<code>run_stage4_outlier_report()</code>	查异常点和极端误差。
<code>check_stage4_npz_fields_current_run()</code>	抽样检查 npz 中字段是否完整。
<code>build_key_log()</code>	汇总关键日志，方便快速查看。
<code>main()</code>	主入口，按计划顺序执行所有阶段。

2. `stage3_agents_v2.py` 中的主要函数

函数	通俗解释
<code>_resolve_stage2_summary_path()</code>	找到 Stage2 的 summary 文件。
<code>_resolve_stage3_output_dir()</code>	决定 Stage3 输出写到哪里。
<code>_linear_idx_from_df()</code>	把三维网格坐标转成一维索引，方便快速查找。
<code>_linear_to_zyx()</code>	把一维索引还原成三维坐标。
<code>_expand_linear_support()</code>	把某些支撑点向周围扩一点，用于邻域支持判断。
<code>_refine_flight_pack_with_wind_support()</code>	根据附近风信息修正 flight agent 的 wind support。
<code>load_stage2_voxel()</code>	读取某一帧的 Stage2 voxel npz。
<code>_safe_float_series()</code>	安全读取浮点列，避免空值或异常格式影响运行。
<code>_robust_center()</code>	计算稳健中心值，减少极端值影响。
<code>_weighted_center()</code>	按权重计算中心值。
<code>_weighted_center_by_flight()</code>	对每架飞机分别计算加权中心。
<code>_count_positive_entries()</code>	统计大于阈值的有效项数量。
<code>_format_duration()</code>	把秒数格式化成易读时间。
<code>_print_stage_progress()</code>	打印 Stage3 进度。
<code>_select_stage2_frames()</code>	根据运行配置选择要处理哪些帧。
<code>_estimate_ground_from_frame()</code>	从当前帧估计地面参考位置。
<code>_diag_frame_distribution()</code>	输出当前帧数据分布诊断信息。

<code>build_agents_for_frame()</code>	Stage3 核心函数：给单帧构建 flight agents 和通信关系。
<code>main()</code>	Stage3 主入口，逐帧构建 agents 并写出 JSON 和 summary。

3. stage4_pack_v2.py 中的核心函数

函数	通俗解释
<code>_resolve_stage2_summary_path()</code>	找 Stage2 summary。
<code>_resolve_stage3_input_dir()</code>	找 Stage3 输入目录。
<code>_resolve_stage4_output_dir()</code>	决定 Stage4 输出目录。
<code>_resolve_stage4_aux_source_dir()</code>	FullAux 模式下找到 fast 源结果目录。
<code>_load_stage3_pack_map()</code>	读取 Stage3 的 agents JSON，并建立帧到 agent pack 的映射。
<code>_json_to_flight_pack()</code>	把 JSON 格式的 agent pack 转回程序内部结构。
<code>_sanitize_observations()</code>	清洗观测表，去掉缺字段或异常值。
<code>_concat_frames()</code>	合并多个数据帧。
<code>_merge_motion_views()</code>	合并不同来源的运动信息。
<code>_choose_wind_comm_source()</code>	在 primary wind、AMDAR、fallback wind 中选择用于通信/重构的风源。
<code>_dedupe_primary_wind_source()</code>	处理 direct wind 和其他风源重叠，避免同一位置重复计权。
<code>_build_direct_source_maps()</code>	建立 direct observation 的坐标映射。
<code>_build_direct_agreement_fields()</code>	计算不同直接观测源之间是否一致。
<code>_restore_direct_anchor_voxels()</code>	恢复被后处理影响的 direct anchor 点。
<code>_enforce_direct_anchor_values()</code>	最终强制 direct anchor 点优先，保护直接观测不被改坏。
<code>_build_trajectory_volume()</code>	把飞行轨迹转成三维体素场。
<code>_flight_pack_to_seed_df()</code>	把 Stage3 agent pack 转成 Stage4 可用的 seed 点。
<code>_build_support_strength()</code>	综合轨迹、motion 和 seed，构建每个体素的 support 强度。
<code>_build_domain_mask()</code>	决定当前帧哪些体素属于可重构区域。
<code>_support_guided_fill()</code>	根据 support 强度补全缺失风场。
<code>_apply_temporal_background()</code>	利用上一帧状态补当前帧，保证时序连续性。
<code>_physics_guided_relaxation()</code>	做轻量物理一致性平滑，让风场局部更稳定。
<code>_refine_recon_confidence()</code>	调整重构置信度，让 direct 区域高、补全区域低，形成层次。
<code>_suppress_recon_outliers()</code>	压制明显异常的重构点。
<code>_prune_low_quality_reconstruction()</code>	裁剪低质量、低置信度区域。
<code>_expand_supported_reconstruction()</code>	在高 support 区域温和扩展 coverage。
<code>_forecast_next_wind_field()</code>	根据当前帧生成下一帧可用的时序背景。
<code>_compute_pinn_proxy_fields()</code>	计算 PINN 训练可用的 divergence / smoothness proxy。
<code>_compute_hazard_proxies()</code>	计算 turbulence、shear 等风险 proxy。

<code>_compute_source_diversity_field()</code>	统计同一位置有多少类来源支撑，表示来源多样性。
<code>_build_diffusion_condition_tensors()</code>	构建后续 diffusion / 预测模型可能用到的条件张量。
<code>_build_confidence_source_fields()</code>	构建 confidence 相关的来源解释字段。
<code>_build_where2comm_targets()</code>	构建 Where2Comm 风格的通信目标，用于协同感知。
<code>_topk_sparse_targets()</code>	从候选目标中选 top-k，减少通信目标数量。
<code>_should_trigger_reconstruction()</code>	判断当前帧是否需要重构，以及触发原因。
<code>_prepare_frame()</code>	Stage4 主重构核心函数：读取一帧输入，融合多源信息，输出完整重构结果。
<code>_build_sparse_stage4_payload()</code>	把重构结果打包成保存用的稀疏/紧凑 payload。
<code>_build_lightweight_frame()</code>	构建轻量帧结构，主要用于不重跑主重构的辅助导出。
<code>_load_stage4_fast_payload()</code>	FullAux 模式下读取 fast 输出 npz。
<code>_build_full_aux_fields_from_fast()</code>	从 fast 结果补算完整辅助场。
<code>_augment_existing_stage4_npz_with_full_aux()</code>	把 fast npz 增强成 full aux npz，不改变 recon_*。
<code>_stage4_should_skip_post()</code>	判断 fast 模式是否跳过部分后处理。
<code>_stage4_should_skip_dense_aux()</code>	判断 fast 模式是否跳过 dense auxiliary fields。
<code>_stage4_profile_support_keep_q()</code>	根据 profile 决定 support 保留分位数。
<code>_stage4_profile_expand_params()</code>	根据 profile 决定 expand 参数。
<code>_stage4_quality_profile_name()</code>	返回当前质量档位名称，例如 fast_balanced 或 aux_aggressive。
<code>_resolve_stage4_gpu_device()</code>	判断 GPU 是否可用，并确定使用哪个设备。
<code>_stage4_gpu_enabled()</code>	根据数组大小和 GPU 状态判断某一步是否走 GPU。
<code>_torch_from_numpy()</code> / <code>_torch_to_numpy()</code>	NumPy 和 Torch 张量之间转换。
<code>_torch_neighbor_avg()</code> / <code>_torch_lap()</code>	GPU 上计算邻域平均和 Laplacian。
<code>_dense_field_to_sparse()</code>	把 dense 三维场转成 sparse idx/value，减少存储。
<code>_inflate_local_field()</code>	把局部子区域结果放回全局网格。
<code>_slice_prev_recon_state()</code>	从上一帧状态里切出当前局部区域需要的部分。
<code>_lift_where2comm_targets()</code>	把局部通信目标坐标恢复到全局坐标。
<code>_masked_mean()</code> / <code>_masked_quantile()</code>	在 mask 区域内统计均值和分位数。
<code>_print_stage_progress()</code>	打印 Stage4 进度。
<code>_select_stage2_frames()</code>	选择 Stage4 要处理的帧。
<code>_build_frame_meta()</code> / <code>_build_stage4_meta_from_summary()</code>	构建帧级元信息，方便写 summary。
<code>_infer_hw_from_vox()</code>	从 voxel 数据中推断网格大小。
<code>_load_stage4_fast_payload()</code>	读取 fast npz 里的核心字段。
<code>main()</code>	Stage4 主入口。fast 模式逐帧重构；FullAux 模式读取 fast 结果并补写辅助场。

4. 汇报时最应该讲的 8 个函数

如果组会时间短，不需要讲所有函数，重点讲下面 8 个就够：

函数	为什么重要
<code>run_full_fast_multi_gpu()</code>	说明当前工程主链路：Stage3 并行，Stage4 单卡串行。
<code>merge_stage3_shards()</code>	说明 Stage3 分片结果如何合并成正式 <code>stage3_output_v2</code> 。
<code>build_agents_for_frame()</code>	说明 Stage3 怎么从每帧数据生成 flight agents 和通信关系。
<code>_prepare_frame()</code>	Stage4 主重构核心，所有多源融合都在这里串起来。
<code>_support_guided_fill()</code>	解释 support fill 如何补全稀疏区域。
<code>_apply_temporal_background()</code>	解释为什么 Stage4 不能随便分片，因为它依赖上一帧。
<code>_restore_direct_anchor_voxels()</code> / <code>_enforce_direct_anchor_values()</code>	解释 direct anchor protection，保证直接观测不被后处理破坏。
<code>_augment_existing_stage4_npz_with_full_aux()</code>	解释 S6 FullAux 为什么不是重跑主链，而是基于 fast 结果补辅助场。

5. 函数调用主线理解版

如果把整个流程看成一条流水线，可以按下面顺序理解：

```
run_stage34_workflow_v2.sh
-> run_full_fast_multi_gpu()
    -> split_indices_into_shards()
    -> run stage3_agents_v2.py on each shard
    -> merge_stage3_shards()
    -> run stage4_pack_v2.py once, single GPU serial

stage3_agents_v2.py
-> main()
    -> load_stage2_voxel()
    -> build_agents_for_frame()
    -> write agents/*.json and stage3_summary.json

stage4_pack_v2.py
-> main()
    -> _prepare_frame()
        -> _build_support_strength()
        -> _support_guided_fill()
        -> _apply_temporal_background()
        -> _physics_guided_relaxation()
        -> _refine_recon_confidence()
        -> _prune_low_quality_reconstruction()
        -> _expand_supported_reconstruction()
        -> _restore_direct_anchor_voxels()
        -> _enforce_direct_anchor_values()
    -> write frame_*.npz and stage4_summary.json
```

S6 FullAux 的流程不一样，它不是重新 `_prepare_frame()`：

```
stage4_pack_v2.py full_aux_export
-> main()
-> _load_stage4_fast_payload()
-> _build_full_aux_fields_from_fast()
-> _augment_existing_stage4_npz_with_full_aux()
-> write full aux frame_*.npz
```

这个区别很关键：

- S5 FinalFast 是主重构。
- S6 FullAux 是基于 S5 结果补辅助字段。
- S6 理论上不应该改变 recon_u_3d / recon_v_3d / recon_confidence_3d / recon_mask_3d。

九、论文实验矩阵

当前已经整理出 S0-S6 内部 baseline / ablation 矩阵。

Version	SUPPORT_FILL	TEMPORAL_FILL	RELAX	PRUNE	EXPAND	ANCHOR_RESTORE	ANCHOR_FORCE
S0 DirectOnly	0	0	0	0	0	0	0
S1 BaseRecon	0	0	0	0	0	0	0
S2 SupportTemporal	1	1	0	0	0	0	0
S3 PhysicsSmooth	1	1	1	0	0	0	0
S4 ConfPruneAnchor	1	1	1	1	0	1	1
S5 FinalFast	1	1	1	1	1	1	1
S6 FullAux	1	1	1	1	1	1	1

实验指标分为四类：

1. Sparse anchor fidelity

- vector_rmse
- corr_u
- corr_v
- outlier_count

2. Full-field reconstruction quality proxies

- recon_conf_mean
- recon_coverage_ratio
- recon_conf_p10/p50/p90
- recon_conf_spread_p10_p90

3. Interpretability / structure indicators

- support_fill_voxels
- temporal_fill_voxels

- support_expand_voxels
- anchor_restore_voxels
- anchor_force_voxels
- physics_weight_mean
- source_diversity_mean

4. Runtime metrics

- total elapsed
- fps
- Stage3 shard policy
- Stage4 single-gpu-serial policy

十、已观察到的阶段性结果

当前记录中的关键基线：

版本	recon_conf_mean_avg	recon_coverage_avg	fullfield_confidence_spread_avg
fast_single_gpu_baseline_v2	约 0.249393	约 0.113614	约 0.474462
fast_single_gpu_baseline_v3_quality	约 0.2719	约 0.1150	待全量确认

阶段性判断：

- anchor fidelity 是当前 Stage4 的强项。
- confidence 可以通过保守提质得到改善。
- coverage 受数据稀疏性限制，不能无限追高。
- GPU 已接入部分 dense 数值核，但整体瓶颈仍包含 CPU、Polars、稀疏索引和 npz I/O，因此不追求整条 Stage4 全搬 GPU。

十一、下周计划

P0：完成 S5 FinalFast 全量主运行

使用推荐命令：

```
BASE_DIR=/data/LFT-W02_data/pengxu
STAGE_DIR=$BASE_DIR/stage

RUN_MODE=full \
RUN_LABEL_OVERRIDE=full_fast_stage4_frozen_v1 \
RUN_PHASE=full_fast_multi_gpu \
RUN_VALIDATE=1 \
PROGRESS_EVERY=50 \
STAGE3_PARALLEL_SHARDS=8 \
STAGE3_CPU_THREADS_PER_WORKER=1 \
STAGE4_CPU_THREADS=6 \
MULTI_GPU_STAGE4_SHARD=0 \
WIND_STAGE4_USE_GPU=1 \
WIND_STAGE4_GPU_DEVICE=cuda:0 \
WIND_STAGE4_ENABLE_SUPPORT_FILL=1 \
WIND_STAGE4_ENABLE_TEMPORAL_FILL=1 \
WIND_STAGE4_ENABLE_RELAX=1 \
WIND_STAGE4_ENABLE_PRUNE=1 \
WIND_STAGE4_ENABLE_EXPAND=1 \
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_RESTORE=1 \
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_FORCE=1 \
bash $STAGE_DIR/run_stage34_workflow_v2.sh
```

P0: 完成 S6 FullAux 训练导出

```
BASE_DIR=/data/LFT-W02_data/pengxu
STAGE_DIR=$BASE_DIR/stage

RUN_MODE=full \
RUN_LABEL_OVERRIDE=full_aux_export_stage4_frozen_v1 \
RUN_PHASE=full_aux_export \
RUN_VALIDATE=1 \
PROGRESS_EVERY=50 \
STAGE4_FAST_SOURCE_DIR=$BASE_DIR/stage4_output_v2 \
WIND_STAGE4_USE_GPU=1 \
WIND_STAGE4_ENABLE_SUPPORT_FILL=1 \
WIND_STAGE4_ENABLE_TEMPORAL_FILL=1 \
WIND_STAGE4_ENABLE_RELAX=1 \
WIND_STAGE4_ENABLE_PRUNE=1 \
WIND_STAGE4_ENABLE_EXPAND=1 \
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_RESTORE=1 \
WIND_STAGE4_ENABLE_DIRECT_ANCHOR_FORCE=1 \
bash $STAGE_DIR/run_stage34_workflow_v2.sh
```

P1: 固定子集完成 S0-S4 消融

先在固定子集上跑，不直接全部全量，降低成本并先确认差异是否稳定。

P1: 整理论文结果表

按以下表格组织：

- Main Result Table
- Module Contribution Table
- Structural Statistics Table
- Fast vs FullAux Table

P2: 准备 Stage5 设计草案

Stage5 不做全域逐帧预测，而是从以下方向展开：

- event-driven trigger
- ROI-focused prediction
- short-term wind-field nowcasting
- hazard-aware decision support

十二、当前待解决问题

1. S5 FinalFast 全量运行结果还需要完整落档。
2. S6 FullAux 需要验证 recon_* 是否完全不变。
3. S0-S4 消融需要先确定固定子集，再决定是否全量。
4. 高重叠帧中 wind_primary 可能过度去重，需要作为结构性分析点。
5. coverage 提升有限，需要在论文中解释为稀疏观测条件下的合理边界。
6. 当前默认链路不跑重报告，论文最终指标需要按需补跑 sparse_metrics、outliers、npz_check。
7. Stage5 尚未正式实现，当前只适合作为下一阶段方向。

十三、组会简短汇报稿

这个项目主要解决航空场景下风场观测稀疏、不连续、来源复杂的问题。我的思路不是直接训练一个端到端预测网络，而是先把飞机轨迹、直接风观测、运动线索、时序背景和通信图融合成一个稳定的三维风场状态层，作为后续预测和风险判断的输入基础。前期我已经完成了从原始航空观测清洗、体素化，到 flight agent 和通信图构建，再到多源风场重建的基础链路。本周主要把 Stage3/Stage4 的主链路进一步收敛到可全量运行和可消融的状态。当前真实运行使用 run_stage34_workflow_v2.sh、stage3_agents_v2.py 和 stage4_pack_v2.py。工程上已经明确 Stage3 可以做 8 路并行分片，而 Stage4 因为依赖前一帧状态，默认保持单进程单卡串行，只用多线程 CPU 加速。Stage4 现在分成两个版本：S5 FinalFast 作为论文主结果和全量运行版本，S6 FullAux 作为训练前完整辅助场导出版本。同时我把 support fill、temporal fill、relax、prune、expand 和 anchor protect 都做成了显式开关，整理出了 S0-S6 的内部 baseline / ablation 矩阵。下周重点是先跑完 S5 全量结果，再做 S6 导出和固定子集消融，之后再进入 Stage5 的事件驱动、ROI 聚焦短时预测设计。